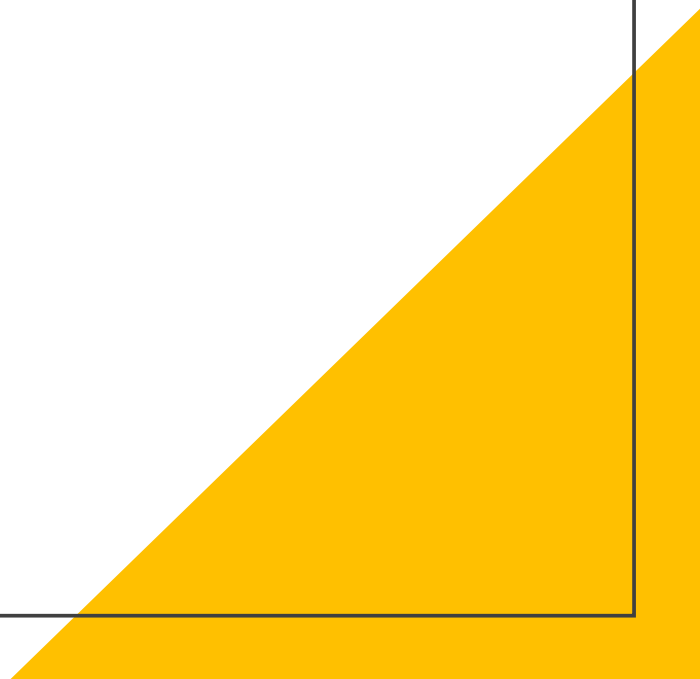


訂閱



訂閱是什麼？

拿大家常用的YouTube來舉例，大家常常看影片會聽到YouTuber再講請各位訂閱小鈴鐺，那為什麼要訂閱呢？

訂閱跟沒訂閱差異就在當YouTuber再更新影片的時候如果有訂閱他，那當他更新影片你就會收到通知，如果你沒訂閱那他更新幾百部你都不會知道也收不到通知。

為什麼要用訂閱?

這時候會請各位回憶一下，我們切換頁面傳遞資料時會新建一個service並且在裡面新增一個變數來存放我們要傳遞的內容，但是接受頁面我們需要一個地方去觸發撈取service中的資料(通常是放在ngOnInit()裡面)，但ngOnInit只會觸發一次當service中的資料再度更新時我們就沒辦法即時去撈取裡面的內容，除非要去新增一個按鈕之類的去做觸發更新，這時候如果我們使用訂閱那只要這個訂閱的**內容**有變更就會觸發訂閱我們就可以直接修改頁面中的內容。

訂閱怎麼寫？

首先訂閱有幾個方法總共有三種分別是

Subject : 單純的訂閱

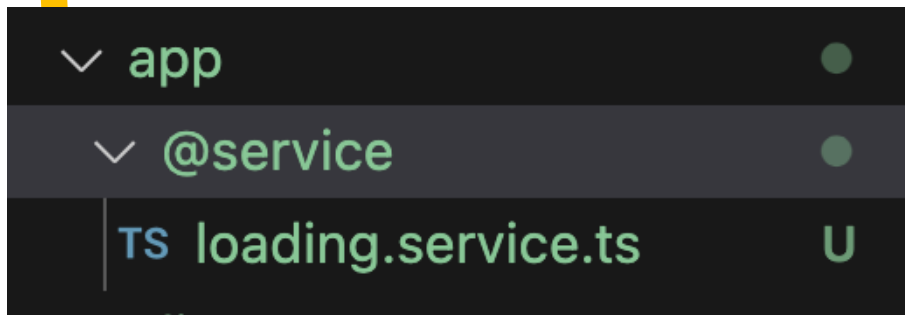
BehaviorSubject : 需要設定初始值的訂閱

ReplaySubject : 當觸發訂閱的時候可以重新發送最後 N 筆資料的訂閱

我們會跟各位介紹Subject與BehaviorSubject 這兩個是最常使用到的訂閱方法。

訂閱怎麼寫？

再來我會建議各位新增一個檔案用來寫訂閱的內容，名稱可以是你要訂閱的欄位或者是訂閱的功用，例如我們之後會教到的loading，你就可以去新增一個檔案叫loading.service.ts用來存放之後loading要用的變數/方法/訂閱內容。



```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class LoadingService {

  constructor() { }

}
```

訂閱怎麼寫？

接著新增一個全域變數並且賦予這個變數為可被訂閱的值(看你要使用有初始值得還是沒有的)，前面的private要注意一下因為要整合訂閱內容變更在同一個地方所以會加private讓這個變數在其他地方不能被使用，這邊我們是設定他的內容是boolean並且初始值為false。

```
// BehaviorSubject可設定初始值
private loading$ = new BehaviorSubject<boolean>(false);
// Subject不需要初始值
private loading2$ = new Subject<boolean>();
```

loading.service.ts

訂閱怎麼寫？

剛剛有新增兩個可被訂閱的變數，但是我們希望他只能在自己的TS被使用，所以我們還需要新增兩個變數來讓其他TS可以去抓取訂閱的內容。

```
// 取得loading$的可觀察物件  
// 取得loading2$的可觀察物件  
_loading$ = this.loading$.asObservable();  
_loading2$ = this.loading2$.asObservable();
```

loading.service.ts

訂閱怎麼寫？

接著我們就可以在下面寫方法去修改我們訂閱的內容，因為我們設定是boolean所以直接設定兩個方法一個將值改為true一個改成false，要修改訂閱的內容的話你需要使用到next這個方法，要注意不能使用_loading\$去修改內容因為他只可以用來觀察訂閱不能修改。

```
// 關閉loading
hide() {
  this.loading$.next(false);
}

// 開啟loading
show() {
  this.loading$.next(true);
}
```

loading.service.ts

怎麼訂閱？

訂閱寫完我們就要去我們要抓取訂閱內容得TS中去做訂閱，記得你要使用service中的內容時請先在該TS中的建構式中帶入該service。

```
constructor(private loadingService: LoadingService) { }
```

app.component.ts

怎麼訂閱？

緊接著我們就可以去呼叫訂閱的變數並且在後面去寫subscribe去訂閱該內容(通常會寫在ngOnInit在這個頁面一開始就去訂閱該值)，當這個訂閱的內容有所改變你就會收到他的內容(res是自己取名的變數，就是訂閱的內容)。

```
ngOnInit(): void {  
  this.loadingService._loading$.subscribe((res) => {  
    console.log('loading$', res);  
  });  
}
```

app.component.ts

结束

